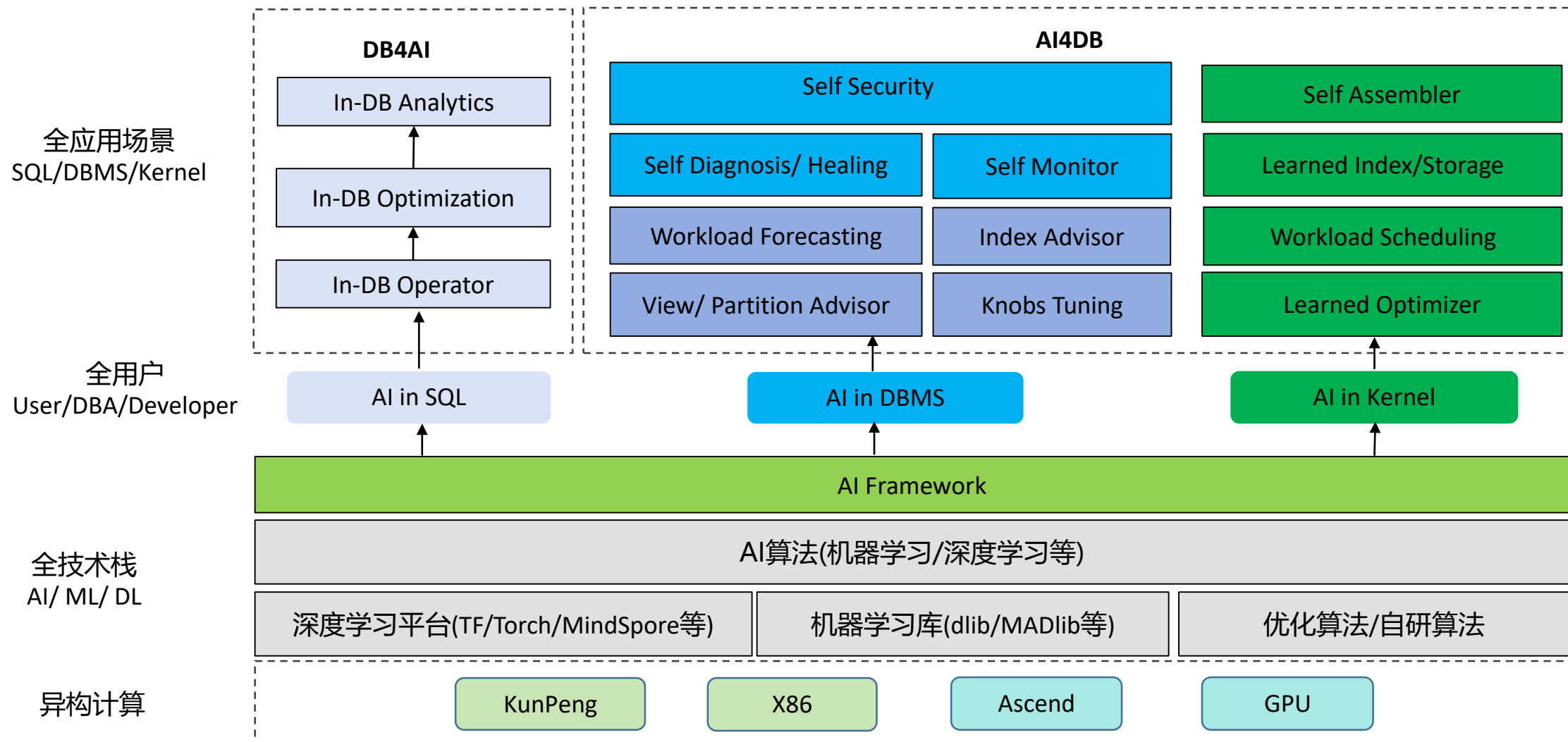




10.3 AI特性

openGauss AI 全景



AI4DB：数据库自治运维

- AI4DB主要用于对数据库进行自治运维和管理，从而帮助数据库运维人员减少运维工作量。在实现上，DBMind的AI4DB框架具有监控和服务化的性质，同时也提供即时AI工具包，提供开箱即用的AI运维功能（如索引推荐）。AI4DB的监控平台以开源的Prometheus为主，DBMind提供监控数据生产者 exporter, 可与Prometheus平台完成对接。

数据库指标采集、预测与异常检测 (1)

- 本特性是openGauss集成的、可以用于数据库指标采集、预测以及异常监控与诊断的AI工具，是DBMind套间中的一个组件。当前通过兼容Prometheus平台来采集数据库系统的指标，提供Prometheus exporter用于采集和加工数据库监控指标。通过监控指标时序数据，可以用来预测未来负载走向，诊断问题，同时还可以进行异常检测等。
- Prometheus是业内非常流行的开源监控系统，同时本身也是一款时序数据库。Prometheus的采集端称之为exporter，用来收集被监控模块的指标项。为了与Prometheus平台完成对接，DBMind分别实现了两款exporter，分别是用来采集数据库指标的openGauss-exporter，以及对采集到的指标进行二次加工的reprocessing-exporter。

数据库指标采集、预测与异常检测 (2)

- **优点：**

- 极大简化运维人员工作，释放大量劳动力，为公司节省成本。
- 用户可以通过指标采集、监控和预测功能提前感知问题，从而防止数据库发生意外，导致更大的损失。

- **约束：**

- 数据库状态正常，并且用户已将数据目录写入环境变量。
- Python 版本要求3.6及以上。
- 配置Prometheus 监控平台，并启动本服务，以便监控数据可被收集。

慢SQL根因分析

- 慢SQL一直是数据运维中的痛点问题，如何有效诊断慢SQL根因是当前一大难题，工具结合openGauss自身特点融合了现网DBA慢SQL诊断经验，支持慢SQL根因分析，能同时按照可能性大小输出多个根因并提供针对性的建议。
- 环境部署
 - 数据库运行正常。
 - 指标采集系统（即openGauss exporter、reprocessing exporter以及node exporter）运行正常。
 - 模块命令行说明：
- 模块命令行说明

```
gs_dbmind component slow_query_diagnosis -help
```

慢SQL根因分析 - 使用指导

- 仅启动慢SQL诊断功能（输出Top3根因），启动命令

```
gs_dbmind service start -c confpath --only-run slow_query_diagnosis
```

- 用户交互式慢SQL诊断，命令如下：

```
gs_dbmind component slow_query_diagnosis show -c confpath --query SQL --start-time timestamps0 --end-time timestamps1
```

- 用户手动清理历史预测结果，命令如下：

- ```
gs_dbmind component slow_query_diagnosis clean -c confpath --retention-days DAYS
```

```
gs_dbmind service stop -c confpath
```

# 慢SQL根因分析 - 常见问题处理

- 如果用户对没有执行过的慢SQL执行交互式诊断命令，则无法给出诊断结果。
- exporter指标采集功能没有启动时运行慢SQL诊断功能，此时功能无法正常运行。
- 配置文件中的参数重新设置后，需要重新启动服务进程才能生效。
- 使用慢SQL的交互诊断功能时，工具基于RPC和数据采集服务获取必要的信息，因此如果RPC和数据采集服务没启动则无法诊断。
- 在使用diagnosis功能进行交互诊断时，工具会对输入的SQL， database进行检测，如果发现属于非法输入则无法诊断。
- 慢SQL诊断中， SMALL\_SHARED\_BUFFER需要采集相关表的字段信息，因此用户需要注意确保opengauss\_exporter连接用户具有该表所在schema的权限。

# Index-advisor 索引推荐

- 索引提供指向存储在表的指定列中的数据值的指针，如同图书的目录，能够加快表的查询速度；
- Index-advisor索引推荐的功能，共包含三个子功能：单query索引推荐、虚拟索引和workload级别索引推荐。

# 单query 索引推荐

- 单query索引推荐功能支持用户在数据库中直接进行操作，单query索引推荐功能基于查询语句的语义信息和数据库的统计信息，对用户输入的单条查询语句生成推荐的索引。
- 单query索引推荐功能涉及的函数接口如下：

| 函数名             | 参数       | 功能              |
|-----------------|----------|-----------------|
| gs_index_advise | SQL语句字符串 | 针对单条查询语句生成推荐索引。 |

# 单query 索引推荐示例 (1)

- 使用gs\_index\_advise函数，获取针对该query生成的推荐索引，推荐结果由索引的表名和列名组成。
  - 示例一：

```
postgres=> select * from gs_index_advise('SELECT c_discount from bmsql_customer where c_w_id = 10');
```

```
table | column
```

```
-----+-----
```

```
bmsql_customer | (c_w_id)
```

```
(1 row)
```

```
CREATE INDEX idx on bmsql_customer(c_w_id);
```

# 单query 索引推荐示例 (2)

- 示例二:

```
postgres=# select * from gs_index_advise('select name, age, sex from t1 where age >= 18 and age < 35 and sex = 'f';');
```

```
table | column
-----+-----
t1 | (age, sex)
(1 row)
```

```
CREATE INDEX idx1 on t1(age, sex);
```

# 虚拟索引

- 虚拟索引功能支持用户在数据库中直接进行操作，虚拟索引功能将模拟真实索引的建立，避免真实索引创建所需的时间和空间开销，用户基于虚拟索引，可通过优化器评估该索引对指定查询语句的代价影响。
- 是否开启虚拟索引功能，虚拟索引功能的GUC参数：enable\_hypo\_index，默认为off。
- 虚拟索引功能涉及的系统函数接口如下表所示：

| 函数名                  | 参数         | 功能说明             |
|----------------------|------------|------------------|
| hypopg_create_index  | 创建索引语句的字符串 | 创建虚拟索引。          |
| hypopg_display_index | 无          | 显示所有创建的虚拟索引信息。   |
| hypopg_drop_index    | 索引的oid     | 删除指定的虚拟索引。       |
| hypopg_reset_index   | 无          | 清除所有虚拟索引。        |
| hypopg_estimate_size | 索引的oid     | 估计指定索引创建所需的空间大小。 |

# 虚拟索引示例 (1)

- 使用函数hypopg\_create\_index创建虚拟索引。例如：

```
postgres=> select * from hypopg_create_index('create index on bmsql_customer(c_w_id)');
indexrelid | indexname
-----+-----
329726 | <329726>btree_bmsql_customer_c_w_id
(1 row)
```

- 开启GUC参数enable\_hypo\_index，该参数控制数据库的优化器进行EXPLAIN时是否考虑创建的虚拟索引。通过对特定的查询语句执行explain，用户可根据优化器给出的执行计划评估该索引是否能够提升该查询语句的执行

```
postgres=> set enable_hypo_index = on;
SET
```

- 开启enable\_hypo\_index后，执行explain，查询计划

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
 QUERY PLAN

Seq Scan on bmsql_customer (cost=0.00..52963.06 rows=31224 width=4)
Filter: (c_w_id = 10)
(2 rows)
```

# 虚拟索引示例 (2)

- 开启GUC参数后，执行EXPLAIN + 查询语句：

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
QUERY PLAN
```

```

[Bypass]
```

```
Index Scan using <329726>btree_bmsql_customer_c_w_id on bmsql_customer (cost=0.00..39678.69 rows=31224 width=4)
```

```
Index Cond: (c_w_id = 10)
```

```
(3 rows)
```

- 通过对比两个执行计划可以观察到，该索引预计会降低指定查询语句的执行代价，用户可考虑创建对应的真实索引。

# workload 级别索引推荐

- 对于workload级别的索引推荐，用户可通过运行数据库外的脚本使用此功能，该功能将包含有多条DML语句的workload作为输入，最终生成一批可对整体workload的执行表现进行优化的索引。
- workload级别索引推荐使用前提：
  - 数据库状态正常、客户端能够正常连接。
  - 当前执行用户下安装有gsqI工具，该工具路径已被加入到PATH环境变量中。
  - 具备Python3.6+的环境。
- 使用步骤：
  - 准备好包含有多条DML语句的文件作为输入的workload，文件中每条语句占据一行。用户可从数据库的离线日志中获得历史的业务语句。
  - 运行python脚本index\_advisor\_workload.py，命令如下：

```
python index_advisor_workload.py [p PORT] [d DATABASE] [f FILE] [--h HOST] [-U USERNAME] [-W PASSWORD] [--max_index_num MAX_INDEX_NUM] [--multi_iter_mode]
```

# workload 级别索引推荐示例

- 将执行多条的DML语句保存在tpcc\_log.txt文件中，执行index\_advisor\_workload.py脚本。

```
python index_advisor_workload.py 6001 postgres tpcc_log.txt --max_index_num 10 --multi_iter_mode
```

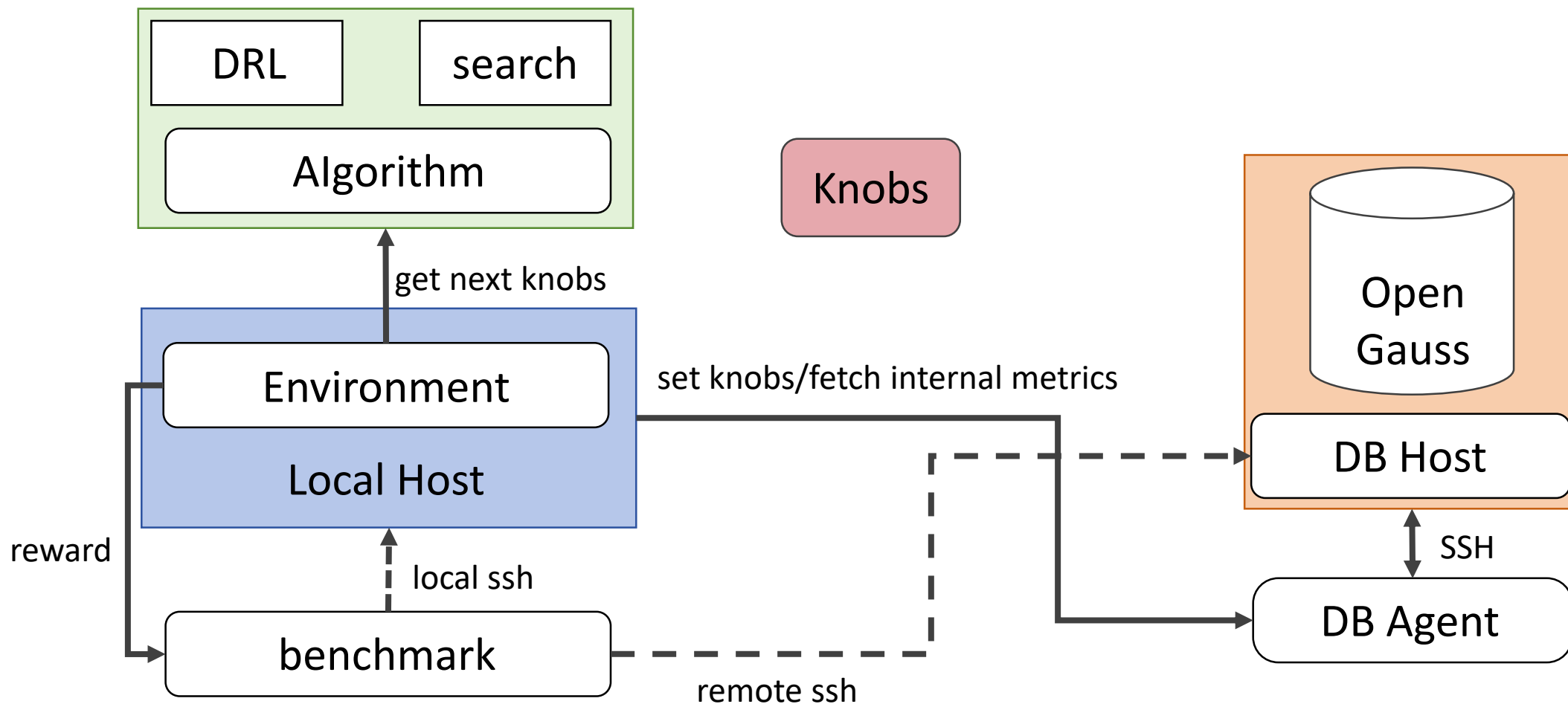
- 推荐结果为一批索引，以多个创建索引语句的格式显示在屏幕上，结果如下：

```
create index ind0 on bmsql_stock(s_i_id,s_w_id);
create index ind1 on bmsql_customer(c_w_id,c_id,c_d_id);
create index ind2 on bmsql_order_line(ol_w_id,ol_o_id,ol_d_id);
create index ind3 on bmsql_item(i_id);
create index ind4 on bmsql_oorder(o_w_id,o_id,o_d_id);
create index ind5 on bmsql_new_order(no_w_id,no_d_id,no_o_id);
create index ind6 on bmsql_customer(c_w_id,c_d_id,c_last,c_first);
create index ind7 on bmsql_new_order(no_w_id);
create index ind8 on bmsql_oorder(o_w_id,o_c_id,o_d_id);
create index ind9 on bmsql_district(d_w_id);
```

# 参数自动调优与诊断

- X-Tuner是一款数据库集成的参数调优工具，通过结合深度强化学习和全局搜索算法等AI技术，实现在无需人工干预的情况下，获取最佳数据库参数配置。本功能不强制与数据库环境部署到一起，支持独立部署，脱离数据库安装环境独立运行。
- 使用调优程序时，需要提供数据库及其所在实例的用户名和登录密码信息，以便控制数据库执行benchmark性能测试；在启动调优程序前，要求用户测试环境交互正常，能够正常跑通benchmark测试脚本、能够正常连接数据库。

# X-Tuner 结构图



# X-Tuner 运行模式

- 调优程序X-Tuner包含三种运行模式，分别是：
  - recommend: 通过用户指定的用户名等信息登录到数据库环境中，获取当前正在运行的workload特征信息，根据上述特征信息生成参数推荐报告。报告当前数据库中不合理的参数配置和潜在风险等；输出根据当前正在运行的workload行为和特征；输出推荐的参数配置。**该模式是秒级的，不涉及数据库的重启操作，其他模式可能需要反复重启数据库。**
  - train: 通过用户提供的benchmark信息，不断地进行参数修改和benchmark的执行。通过反复的迭代过程，训练强化学习模型，以便用户在后面通过tune模式加载该模型进行调优。
  - tune: 使用优化算法进行数据库参数的调优，当前支持两大类算法，一种是深度强化学习，另一种是全局搜索算法（全局优化算法）。深度强化学习模式要求先运行train模式，生成训练后的调优模型，而使用全局搜索算法则不需要提前进行训练，可以直接进行搜索调优。

# X-Tuner 的运行和安装方法 (1)

- 可以通过两种方式运行X-Tuner，一种是直接通过源码运行，另一种则是通过Python的setuptools将X-Tuner安装到系统上，而后直接通过 `gs_xtuner` 命令调用。

- 方法一：直接通过源代码运行

- 切换到xtuner的源代码目录下。对于openGauss社区代码来说，该路径在openGauss-server/src/gausskernel/dbmind/tools/xtuner。对于已经安装的数据库系统，则该源代码路径为 `$GAUSSHOM/bin/dbmind/xtuner`。
- 在当前目录下可以看到 `requirements.txt` 等文件，通过pip包管理工具根据该 `requirements.txt` 文件安装依赖：

```
pip install -r requirements.txt
```

- 安装成功后需要添加环境变量PYTHONPATH，然后可执行main.py. 以获取帮助信息为例，则可以执行：

```
cd tuner # 切换到 main.py 入口文件所在的目录
export PYTHONPATH='..' # 将上一级目录添加到寻找包的路径中
python main.py --help # 可以直接通过该命令执行获取帮助的动作，其他功能使用方法类似
```

# X-Tuner 的运行和安装方法 (2)

- 方法二：将X-Tuner 安装到系统中
  - 可以通过 setup.py 文件将X-Tuner安装到系统中，然后通过命令gs\_xtuner运行。首先需要切换到xtuner的根目录，该路径在openGauss-server/src/gausskernel/dbmind/tools/xtuner。对于已经安装的数据库系统，则该源代码路径为 \$GAUSSHOME/bin/dbmind/xtuner。
  - 执行下述命令，Python 会通过自身的setuptools 将本工具安装到Python环境中：
  - 如果Python的bin目录被添加到PATH环境变量中，则gs\_xtuner命令也可以在任何地方被直接调用。
  - python setup.py install
  - 以获取帮助信息为例，执行下述命令：

```
gs_xtuner --help
```

# 慢SQL发现

- SQLdiag是openGauss中SQL语句执行时长预测工具。现有的预测技术主要基于执行计划的预测方法，但这些预测方案仅适用于OLAP场景且可以获取执行计划的任务，对于OLTP或者HTAP这样的快速、简单查询是没有太多使用价值的。与上述方案不同，SQLdiag着眼于数据库的历史SQL语句，通过对历史SQL语句的执行表现进行总结归纳，将之再用于推断新的未知业务上。

# 慢SQL - 优点

- 不需要SQL语句的执行计划，对数据库性能不会有任何的影响。
- 使用场景广泛，目前业内的很多算法局限性比较高，比如只适用于OLTP或者OLAP，而SQLdiag使用场景广泛。
- 该工具容易理解，只需要简单的操作，就可以训练出自己的预测模型。
- 本工具的典型应用场景是对一批即将上线的SQL语句进行透视，提前识别风险。

# 慢SQL - 前提条件

- 需要保证用户提供训练数据，训练数据格式如下，每个样本通过换行符分隔：

```
SQL,EXECUTION_TIME
```

- 如果用户通过提供的工具收集训练数据，则需要启用WDR功能，涉及到的参数为track\_stmt\_stat\_level和log\_min\_duration\_statement，具体情况见下面小结。
- 为保证预测准确率，用户提供的历史语句日志应尽可能全面并具有代表性。

# 慢SQL - 操作步骤

- 提供历史日志以供模型训练
- 进行训练与预测操作。
  - 基于模板法的训练与预测：

```
gs_dbmind component sqldiag [train, predict] -f FILE --model template --model-path template_model_path
```

```
gs_dbmind component sqldiag [train, predict] -f FILE --model dnn --model-path dnn_model_path
```

# 原生DB4AI引擎

- openGauss当前版本支持了原生DB4AI能力，通过引入原生AI算子，简化操作流程，充分利用数据库优化器、执行器的优化与执行能力，获得高性能的数据库内模型训练能力。更简化的模型训练与预测流程、更高的性能表现，让开发者在更短时间内能更专注于模型的调优与数据分析上，而避免了碎片化的技术栈与冗余的代码实现。

# 原生DB4AI引擎 - 语法及关键字

| 名称  | 描述           |                  |
|-----|--------------|------------------|
| 语法  | CREATE MODEL | 创建模型并进行训练，同时保存模型 |
|     | PREDICT BY   | 利用已有模型进行推断       |
|     | DROP MODEL   | 删除模型             |
| 关键字 | TARGET       | 训练/推断任务的目标列名     |
|     | FEATURES     | 训练/推断任务的数据特征列名   |
|     | MODEL        | 训练任务的模型名称        |

# 原生DB4AI引擎 - 训练模型



# 创建模型

- CREATE MODEL
- 使用“CREATE MODEL”语句可以进行模型的创建和训练。模型训练SQL语句，选用公开数据集鸢尾花数据集iris。
- 以 multiclass 为例，训练一个模型。从 tb\_iris 训练集中指定 sepal\_length, sepal\_width, petal\_length, petal\_width 为特征列，使用 multiclass 算法，创建并保存模型 iris\_classification\_model。

```
openGauss=# CREATE MODEL iris_classification_model USING xgboost_regression_logistic FEATURES sepal_length,
sepal_width, petal_length, petal_width TARGET target_type < 2 FROM tb_iris_1 WITH nthread=4, max_depth=8;
MODEL CREATED. PROCESSED 1
```

# 查看模型信息 (1)

- 当训练完成后模型会被存储到系统表gs\_model\_warehouse中。系统表gs\_model\_warehouse可以查看到关于模型本身和训练过程的相关信息。
- 关于模型的详细描述信息以二进制的形式存储在系统表中，用户可用过使用函数gs\_explain\_model完成对模型的查看，语句如下：

```
openGauss=# select * from gs_explain_model("iris_classification_model");
DB4AI MODEL

Name: iris_classification_model
Algorithm: xgboost_regression_logistic
Query: CREATE MODEL iris_classification_model
USING xgboost_regression_logistic
FEATURES sepal_length, sepal_width,petal_length,petal_width
TARGET target_type < 2
FROM tb_iris_1
WITH nthread=4, max_depth=8;
```

# 查看模型信息 (2)

```
Return type: Float64
Pre-processing time: 0.000000
Execution time: 0.001443
Processed tuples: 78
Discarded tuples: 0
n_iter: 10
batch_size: 10000
max_depth: 8
min_child_weight: 1
gamma: 0.0000000000
eta: 0.3000000000
nthread: 4
verbosity: 1
seed: 0
booster: gbtrees
tree_method: auto
eval_metric: rmse
rmse: 0.2648450136
model size: 4613
```

# 推断任务

- 利用已存在的模型做推断任务。
- 使用 “SELECT” 和 “PREDICT BY” 关键字利用已有模型完成推断任务。
- 查询语法：SELECT...PREDICT BY...(FEATURES...)...FROM...;

```
openGauss=# SELECT id, PREDICT BY iris_classification (FEATURES sepal_length,sepal_width,petal_length,petal_width) as "PREDICT" FROM
tb_iris limit 3;
id | PREDICT
----+-----
 84 | 2
 85 | 0
 86 | 0
(3 rows)
```

# 查看执行计划

- 查看执行计划。
- 使用explain语句可对“CREATE MODEL”和“PREDICT BY”的模型训练或预测过程中的执行计划进行分析。Explain关键字后可直接拼接CREATE MODEL/ PREDICT BY语句（子句），也可接可选的参数，支持的参数如下：

```
openGauss=# Explain CREATE MODEL patient_logisitic_regression USING logistic_regression FEATURES second_attack, treatment TARGET
trait_anxiety > 50 FROM patients WITH batch_size=10, learning_rate = 0.05;
 QUERY PLAN
```

```

Train Model - logistic_regression (cost=0.00..0.00 rows=0 width=0)
-> Materialize (cost=0.00..41.08 rows=1776 width=12)
 -> Seq Scan on patients (cost=0.00..32.20 rows=1776 width=12)
(3 rows)
```

# 模型解析

- 模型解析。
- 场景：当模型名在系统表中查找不到，数据库会报ERROR

```
openGauss=# select gs_explain_model("ecoli_svmc");
ERROR: column "ecoli_svmc" does not exist
```

# 异常场景

- 训练阶段

- 场景一：当超参数的设置超出取值范围，模型训练失败，返回ERROR，并提示错误
- 场景二：当模型名称已存在，模型保存失败
- 场景三：FEATURE或者TARGETS列是\*，
- 场景四：对于无监督学习方法使用TARGET关键字，或者在监督学习方法中不适用TARGET关键字，
- 场景五：当进行分类任务时TARGET列的分类只有1种情况，
- 场景六：DB4AI在训练过程中会过滤掉含有空值的数据，当参与训练的模型数据为空的时候

# DB4AI-Snapshots数据版本管理 (1)

- 概念

- DB4AI-Snapshots是DB4AI模块用于管理数据集版本的功能。通过DB4AI-Snapshots组件，开发者可以简单、快速地进行特征筛选、类型转换等数据预处理操作，同时还可以像git一样对训练数据集进行版本控制。数据表快照创建成功后可以像视图一样进行使用，但是一经发布后，数据表快照便固化为不可变的静态数据，如需修改该数据表快照的内容，需要创建一个版本号不同的新数据表快照。

# DB4AI-Snapshots数据版本管理 (2)

- 生命周期

- DB4AI-Snapshots的状态包括published、archived以及purged。其中，published可以用于标记该DB4AI-Snapshots 已经发布，可以进行使用。archived表示当前 DB4AI-Snapshots 处于“存档期”，一般不进行新模型的训练，而是利用旧数据对新的模型进行验证。purged则是该DB4AI-Snapshots 已经被删除的状态，在数据库系统中无法再检索到。
- 需要注意的是快照管理功能是为了给用户提供统一的训练数据，不同团队成员可以使用给定的训练数据来重新训练机器学习模型，方便用户间协同。为此**私有用户**和**三权分立**状态(enableSeparationOfDuty=ON)等涉及不支持用户数据转写等情况将不支持Snapshot特性。

# DB4AI-Snapshots数据版本管理 (3)

- 用户可以通过“CREATE SNAPSHOT”语句创建数据表快照，创建好的快照默认即为published状态。可以采用两种模式创建数据表快照，即为MSS以及CSS模式，它们可以通过GUC参数db4ai\_snapshot\_mode进行配置。可以通过“ARCHIVE SNAPSHOT”语句将某一个数据表快照标记为archived状态，可以通过“PUBLISH SNAPSHOT”语句将其再度标记为published状态。标记数据表快照的状态，是为了帮助数据科学家进行团队合作使用的。
- 当一个数据表快照已经丧失存在价值时，可以通过“PURGE SNAPSHOT”语句删除它，以便永久删除其数据并恢复存储空间。

# DB4AI-Snapshots数据版本管理 (4)

- 使用指导

- 创建表以及插入表数据。

- 数据库内存在已有的数据表，可根据该已有的数据表创建对应的数据表快照。为了后续演示，在此处新建一个名为 t1 的数据表，并向其中插入测试数据。查询搭配数据表内容。

- 使用DB4AI-Snapshots。

- 创建DB4AI-Snapshots：CREATE SNAPSHOT...AS

- 示例1：示例如下，其中，默认版本分隔符为 “@”，默认子版本分割符为 “.”，该分割符可以分别通过GUC参数 “db4ai\_snapshot\_version\_delimiter” 以及 “db4ai\_snapshot\_version\_separator” 进行设置。

# DB4AI-Snapshots数据版本管理 (5)

```
create snapshot s1@1.0 comment is 'first version' as select * from t1;
```

```
schema | name
```

```
-----+-----
```

```
public | s1@1.0
```

```
(1 row)
```

样进行查询，但不支持通过 “INSERT INTO” 语句进行更新。

# DB4AI-Snapshots数据版本管理 (6)

- 例如下面几种语句都可以查询到数据表快照s1的对应版本 1.0的内容：

```
SELECT * FROM s1@1.0;
SELECT * FROM public.s1@1.0;
SELECT * FROM public . s1 @ 1.0;
```

```
id | name
```

```
----+-----
```

```
1 | zhangsan
```

```
2 | lisi
```

```
3 | wangwu
```

```
4 | lisa
```

```
5 | jack
```

```
(5 rows)
```

# DB4AI-Snapshots数据版本管理 (7)

- 可以通过下列SQL语句修改数据表t1的内容：

```
UPDATE t1 SET name = 'tom' where id = 4;
insert into t1 values (6, 'john');
insert into t1 values (7, 'tim');
```

吉

果并未发生变化。由于数据表t1的数据已经发生了改变，如果将当前数据表的内容作为版本2.0，则可创建快照**s1@2.0**,创建的SQL语句如下：

```
create snapshot s1@2.0 as select * from t1;
```

# DB4AI-Snapshots数据版本管理 (8)

- 删除数据表快照SNAPSHOT

```
purge snapshot s1@3.0;
schema | name
-----+-----
public | s1@3.0
(1 row)
```

# DB4AI-Snapshots数据版本管理 (9)

- 从数据表快照中采样
  - 示例：从snapshot s1中抽取数据，使用0.5抽样率。

```
sample snapshot s1@2.0 stratify by name as nick at ratio .5;
schema | name
-----+-----
public | s1nick@2.0
(1 row)
```

- 可以利用该功能创建训练集与测试集，例如：

```
SAMPLE SNAPSHOT s1@2.0 STRATIFY BY name AS _test AT RATIO .2, AS _train AT RATIO .8 COMMENT IS 'training';
schema | name
-----+-----
public | s1_test@2.0
public | s1_train@2.0
(2 rows)
```

# DB4AI-Snapshots数据版本管理 (10)

- 发布数据表快照
  - 采用下述SQL语句将数据表快照 s1@2.0 标记为published 状态:

```
publish snapshot s1@2.0;
```

```
schema | name
```

```
-----+-----
```

```
public | s1@2.0
```

```
(1 row)
```

# DB4AI-Snapshots数据版本管理 (11)

- 存档数据表快照
  - 采用下述语句可以将数据表快照标记为 archived 状态：

```
archive snapshot s1@2.0;
schema | name
-----+-----
public | s1@2.0
(1 row)
```

您可以通过db4ai-snapshots提供的视图查看当前的数据表快照的状态以及其他信息。

```
select * from db4ai.snapshot
```

# DB4AI-Snapshots数据版本管理(12)

- 相关GUC参数

- db4ai\_snapshot\_mode:

- Snapshot有2种模式：MSS（物化模式，存储数据实体）和CSS（计算模式，存储增量信息）。Snapshot可在MSS和CSS之间切换快照模式，默认是MSS模式。

db4ai\_snapshot\_version\_delimiter:

- 该参数为数据表快照子版本分隔符。 "." 为数据表快照的默认版本分隔符。

db4ai\_snapshot\_version\_separator

# ABO优化器

- 智能基数估计

- 智能基数估计利用库内轻量级算法进行多列数据分布建模，并且提供多列等值基数估计的能力。在数据分布倾斜并且列之间相关性强的数据场景下能够提供更准确的估计结果，从而给优化器提供准确的代价参考，提高计划生成准确率，提高数据库查询执行效率。

- 自适应计划选择

- 本特性通过触发基于基表条件选择率的计划选择，以及对于使用了部分索引和offset的查询提供缓存多计划管理和自适应选择。典型场景下能够提升数倍查询吞吐。自适应计划选择作用于使用通用缓存计划进行计划执行的场景。通过使用范围线性扩张进行缓存计划探索，通过范围覆盖匹配进行计划选择。自适应计划选择弥补了传统单一缓存计划无法根据查询条件参数进行变化带来带来的性能问题，并且避免了频繁调用查询优化。